

PROJE 13 / 20 · ÇALIŞAN TEMEL SÜRÜM

Randevu ve Müsaitlik Takvimi

Tek uzmanın 30 dakikalık saatlerini ayır; aynı slotun iki kullanıcıya verilmesini önle.

İstanbul saati · tek uzman

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.

DYFULL-STACK LAB / 13 / 20Deniz DemoÇıkış

PORTFOLYO PROJESİ · ÇALIŞAN TEMEL SÜRÜM

Randevu ve Müsaitlik Takvimi

Tek uzmanın 30 dakikalık saatlerini ayır; aynı slotun iki kullanıcıya verilmesini önle.

İstanbul saati · tek uzman

Randevu seç

Tek uzman · 09:00-17:00 · 30 dakika · Europe/Istanbul (UTC+03:00)

Ad soyad

Gün

09:00 · Uygun

09:30 · Uygun

10:00 · Uygun

10:30 · Uygun

11:00 · Uygun

11:30 · Uygun

Randevularım

Henüz randevun yok.

Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

Gün ve slot seçimi · Müsaitlik listesi · Gelecek tarih kontrolü · Randevu iptali · Aktif slot için UNIQUE indeks

Bu sürümde olmayanlar

Sabit 09:00-17:00 ve 30 dakika; hafta sonu/tatil hariç tutma yok. İstanbul için UTC+03 sabit ofset kullanılır; genel timezone/DST motoru değildir.

DENİZ YARDIMCI · FULL-STACK LAB · 23.09.2026

1

02 / Backend ve veri modeli

appointments tablosunda status=booked koşullu UNIQUE(date,slot) indeksi aynı aktif slotu tekilleştirir. İptal kaydı silmez, cancelled yapar ve slot tekrar alınabilir. API tüm gün/saat değerlerini doğrular; geçmiş randevu reddedilir. Kullanıcı yalnız kendi randevusunu iptal eder.

Gerçek SQL şeması

```
CREATE TABLE appointments(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),date TEXT,slot TEXT,name TEXT,status TEXT DEFAULT 'booked');
```

```
CREATE UNIQUE INDEX active_slot ON appointments(date,slot) WHERE status='booked';
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekiliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametrelili SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/slots	Query: date	200
GET /api/appointments	Gövde yok	200
POST /api/appointments	JSON: date, slot, name	201
DELETE /api/appointments/:id	Gövde yok	200

Somut istek örneği

```
POST /api/appointments
{"date":"2026-10-01","slot":"09:30","name":"Deniz"}
201 {"id":"...", "startsAt":"2026-10-01T09:30:00+03:00"}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- Eşzamanlı aynı slot isteğinde yalnız biri başarılı olmalı.
- İptal slotu yeniden açmalı.
- Geçmiş gün/saat reddedilmeli.
- Başka kullanıcının randevusu iptal edilememeli.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const date=new Date(Date.now()+86400000*2).toISOString().slice(0,10);
const responses=await Promise.all([raw('/api/appointments','POST',{date:slot:'09:00',name:'One'},cookie),raw('/api/appointments','POST',{date:slot:'09:00',name:'Two'},other)]);
assert.deepEqual(responses.map(x=>x.status).sort(),[201,409]);
const w=responses[0].status===201?cookie:other;const a=await
responses.find(x=>x.status===201).json();
await call('/api/appointments/'+a.id,'DELETE',undefined,200,w);
assert.equal((await
get('/slots?date='+date)).slots.find(x=>x.time==='09:00').available,true);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

İki hesapla aynı gelecek slotu eşzamanlı iste. Başarılı randevuyu iptal et ve slotun tekrar müsait olduğunu göster.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Uzman ve hizmet süresi modelleri ekle
- 2. Tatil ve mola saatleri ekle
- 3. IANA timezone/DST desteği ekle
- 4. PostgreSQL aralık çakışması kısıtına geç

Anlamli commit sırası

```
git add .
git commit -m "chore: add appointment-booking baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

30 ve 45 dakikalık hizmetler birlikte sunulursa sabit slot UNIQUE kuralı yeterli olur mu?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.